

Laporan Praktikum M03 – Optimizer dan Strategi Pelatihan

Modul Praktikum Deep Learning

Nama	Natasya Amavisca	NIM	123450024	Kelas	Deep Learning RB
Modul	M03 – Optimizer dan Strategi Pelatihan	Tanggal praktikum	2026-09-30	Seed / device	24 / CPU
Berkas:	M03_123450024.ipynb, M03_123450024_metrics.csv				
	Run: 19, seluruhnya status=ok				
	Device: CPU, PyTorch 2.14.0+cpu				

Ringkasan Singkat

Praktikum ini membandingkan SGD, Momentum, dan Adam (tiga learning rate masing-masing), dua scheduler, dan tiga ukuran batch pada FNN $784 \rightarrow 128 \rightarrow 10$ (101.770 parameter) untuk Fashion-MNIST dengan split 12.000 latih / 3.000 validasi, 5 epoch, batch 128, dan seed 24 di CPU. Kesembilan belas run tercatat pada `metrics.csv`, seluruhnya berstatus ok, dengan total pelatihan ± 31 detik. Adam $\eta = 0,001$ memperoleh validation loss terendah 0,3961 dan validation accuracy 0,8560, mengungguli Momentum (0,4118) dan SGD (0,4263); konfigurasi itu lalu diuji tepat satu kali pada test set (test loss 0,4270; test accuracy 0,8498). Pada anggaran epoch yang sama, scheduler justru menurunkan hasil (StepLR 0,4047; CosineAnnealingLR 0,4066), sedangkan batch 32 unggul tipis (0,3873; 1,875 update) tetapi 2,9 kali lebih lambat per epoch daripada batch 512 (0,4477; 120 update). Karena itu konfigurasi akhir yang direkomendasikan adalah Adam $\eta = 0,001$, batch 128, tanpa scheduler.

1. Tujuan dan Hipotesis

1.1 Tujuan

1) Membandingkan SGD, Momentum, dan Adam pada learning rate masing-masing lalu memilih konfigurasi terbaik dari validation loss; 2) menguji pengaruh StepLR dan CosineAnnealingLR pada anggaran epoch yang sama; 3) menganalisis pengaruh batch size (32, 128, 512) terhadap jumlah update, validation loss, dan waktu per epoch; 4) memverifikasi jumlah update $[n_batch] \times \text{epoch}$ dan mengevaluasi model final pada test set tepat satu kali.

1.2 Hipotesis sebelum eksperimen

Perbandingan	Prediksi	Alasan teknis
Adam vs Momentum vs SGD (validation loss)	Adam < Momentum < SGD	Adam menormalisasi gradien dengan akar rata-rata kuadrat gradien masa lalu sehingga langkah tidak bergantung pada skala gradien dan konvergen cepat pada anggaran pendek.
Learning rate terbaik	Adam menang di sekitar 10^{-3} ; SGD di sekitar 10^{-1}	Ketiganya memakai skala langkah efektif yang berbeda, sehingga learning rate yang sama tidak setara antar optimizer.
Batch 32 vs 512: validation loss	Batch 32 lebih rendah ($\pm 0,38-0,40$)	Pada epoch yang sama batch 32 memberi 1.875 langkah optimizer berbanding 120 langkah, sehingga batch 512 diperkirakan belum konvergen.
Batch 32 vs 512: waktu per epoch	Batch 512 lebih cepat 2-3 kali	Hanya 24 iterasi per epoch berbanding 375; overhead pemanggilan loop jauh lebih sedikit pada batch besar.

Run scheduler (StepLR, CosineAnnealingLR) bersifat eksploratif karena modul tidak meminta prediksi arah sebelum hasil dilihat.

2. Data dan Protokol Eksperimen

2.1 Dataset dan split

Komponen	Nilai
Dataset	Fashion-MNIST (torchvision), 10 kelas busana, citra 28×28 grayscale
Train / val / test	12.000 (20% train resmi) / 3.000 (5%) / 10.000 (test resmi, dipakai tepat satu kali)
Cara split	Terstratifikasi (train_test_split, random_state=24); 1.200 contoh per kelas
Praproses utama	Skala [0, 1] (dibagi 255), lalu $(x - 0,285071)/0,352218$

Mean dan standar deviasi dihitung hanya dari 12.000 contoh latih dan test set dievaluasi tepat satu kali setelah konfigurasi dipilih dari validation loss, sehingga tidak ada kebocoran data.

2.2 Konfigurasi yang dikendalikan

Komponen	Nilai yang digunakan
Arsitektur dasar	FNN Flatten→Linear(784,128)→ReLU→Linear(128,10), 101.770 parameter
Loss & optimizer	CrossEntropyLoss; SGD, SGD + momentum 0.9, atau Adam
Learning rate	SGD {0.001; 0.01; 0.1; 0.5; 1.0}; Momentum {0.01; 0.05; 0.1}; Adam {0.0001; 0.001; 0.01}
Batch size & epoch	128 (470 update); 32 (1.875); 512 (120); 5 epoch
Seed	24, dipanggil ulang (<code>seed_everything</code>) sebelum setiap run
Kriteria pemilihan	Validation loss terendah; test set tidak dipakai untuk memilih
Batas komputasi	CPU; 19 run × 5 epoch; total ±31 detik pelatihan

Dibuat sama: seed, arsitektur, loss, split, praproses, epoch, dan fungsi jalankan() untuk seluruh run. **Diubah:** learning rate, optimizer, scheduler, batch size.
Lingkungan: Python 3.13.13, PyTorch 2.14.0+cpu, NumPy 2.5.1, pandas 3.0.5, device CPU, runtime lokal (JupyterLab).

3. Implementasi dan Pemeriksaan Kebenaran

Pemeriksaan	Nilai yang diharapkan	Hasil aktual	Status
Jumlah parameter model	101.770	101.770 (assert lolos)	Lulus
Ukuran split	12.000 / 3.000 / 10.000	12.000 / 3.000 / 10.000	Lulus
Distribusi kelas latih	1.200 per kelas (terstratifikasi)	1.200 pada 10 kelas	Lulus
Update batch 128	$\lceil 12000/128 \rceil = 94$; 470/5 epoch	470 update tercatat	Lulus
Update batch 32 dan 512	1.875 dan 120	1.875 dan 120; <code>cocok=True</code>	Lulus
Statistik normalisasi dari data latih	mean 0,285071; std 0,352218	identik (dihitung dari latih)	Lulus
Pemakaian test set	tepat satu kali	sekali: loss 0,4270, acc 0,8498	Lulus

Temuan: ketujuh pemeriksaan sesuai harapan, sehingga selisih hasil antar run berasal dari faktor yang sengaja diubah (learning rate, optimizer, scheduler, batch size), bukan dari perbedaan arsitektur, split, atau preproses; kolom `n_update` cocok persis dengan $\lceil n/\text{batch} \rceil \times 5$.

4. Hasil Eksperimen

4.1 Tabel hasil utama

Run	Opt.	LR	n upd	Train loss	Val. loss	Val. acc	s/epoch
<i>Baseline SGD, tiga learning rate (bagian C)</i>							
baseline_sgd_lr0.001	SGD	0,001	470	1,4221	1,3709	0,6303	0,399
baseline_sgd_lr0.1	SGD	0,1	470	0,3933	0,4263	0,8477	0,356
baseline_sgd_lr1	SGD	1,0	470	1,7838	2,3389	0,3330	0,276
<i>Grid 9 run: optimizer $\times$ learning rate (bagian D)</i>							
grid_sgd_lr0.01	SGD	0,01	470	0,6097	0,6183	0,7793	0,412
grid_sgd_lr0.1	SGD	0,1	470	0,3933	0,4263	0,8477	0,226
grid_sgd_lr0.5	SGD	0,5	470	0,4670	0,5676	0,7873	0,304
grid_momentum_lr0.01	Mom.	0,01	470	0,3818	0,4161	0,8490	0,249
grid_momentum_lr0.05	Mom.	0,05	470	0,3144	0,4118	0,8533	0,245
grid_momentum_lr0.1	Mom.	0,1	470	0,3642	0,4621	0,8500	0,245
grid_adam_lr0.0001	Adam	0,0001	470	0,5284	0,5438	0,8073	0,404
grid_adam_lr0.001	Adam	0,001	470	0,3466	0,3961	0,8560	0,283
grid_adam_lr0.01	Adam	0,01	470	0,3520	0,4794	0,8500	0,280
<i>Retrain final + evaluasi test sekali (bagian E)</i>							
final_retrain_adam_lr0.001	Adam	0,001	470	0,3466	0,3961	0,8560	0,262
<i>Scheduler pada batch 128 (bagian F)</i>							
scheduler_none_adam_lr0.001	Adam	0,001	470	0,3466	0,3961	0,8560	0,442
scheduler_step_adam_lr0.001	Adam	0,001 $\rightarrow$ 0,00025	470	0,3495	0,4047	0,8567	0,314
scheduler_cosine_adam_lr0.001	Adam	0,001 $\rightarrow$ 0	470	0,3466	0,4066	0,8550	0,427
<i>Batch size tanpa scheduler (bagian F dan Latihan)</i>							
batch_32_adam_lr0.001	Adam	0,001	1.875	0,3134	0,3873	0,8633	0,608
batch_128_adam_lr0.001	Adam	0,001	470	0,3466	0,3961	0,8560	0,331
batch_512_adam_lr0.001	Adam	0,001	120	0,4106	0,4477	0,8400	0,212

Table 1. Seluruh 19 run pada M03_123450024_metrics.csv (seed 24, 5 epoch, batch 128 kecuali pada eksperimen batch); semua run ok, tidak ada run gagal. Evaluasi test dilakukan sekali pada final_retrain_adam_lr0.001: test loss 0.4270, test accuracy 0.8498. Hanya satu seed (24), sehingga ringkasan mean $\pm$ SD tidak tersedia.

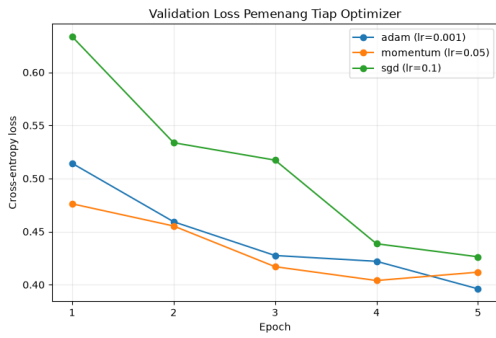


Figure 1: Validation loss per epoch (validation set 3.000 contoh) bagi tiga optimizer terbaik.

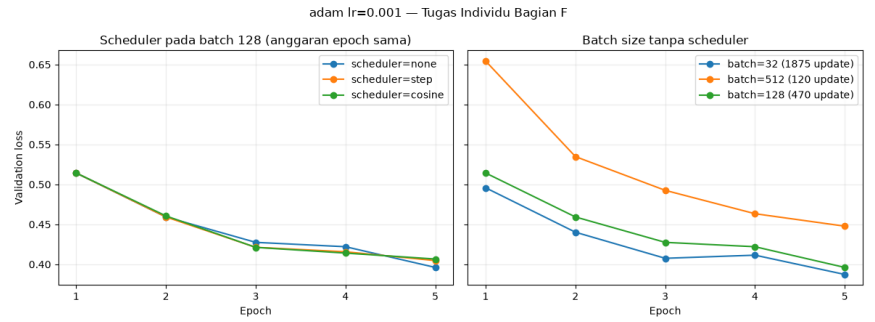


Figure 2: Kiri: scheduler pada batch 128, anggaran epoch sama. Kanan: batch size tanpa scheduler; legenda memuat jumlah update.

4.2 Grafik atau visualisasi utama

Gambar 1: ketiga optimizer turun dari 0,48–0,63 (epoch 1) menuju 0,3961 (Adam), 0,4118 (Momentum), 0,4263 (SGD) pada epoch 5; SGD memulai paling buruk (0,633) tetapi selisihnya terhadap Adam menyusut menjadi 0,030. **Gambar 2 kiri:** kurva hampir berimpit sampai epoch 2 dan baru berpisah setelah learning rate dipangkas; tanpa scheduler tetap terendah (0,3961). **Gambar 2 kanan:** batch 512 memulai 0,655 (epoch 1) dan tertinggal hingga 0,4477, sedangkan batch 32 dan 128 berdekatan (0,3873 vs 0,3961).

5. Analisis dan Pembahasan

Unsur	Temuan	Bukti dan penalaran
Perbandingan dengan hipotesis	Didukung: urutan Adam < Momentum < SGD dan skala learning rate terbaik sesuai prediksi.	Tabel 1: $0,3961 < 0,4118 < 0,4263$; learning rate terbaik Adam 10^{-3} , Momentum 0,05, SGD 10^{-1} . Adam membagi gradien dengan $\sqrt{v}$ sehingga langkah efektif per parameter $\approx \eta$ terlepas dari skala gradien.
Run terbaik dan terburuk	Terbaik <code>grid_adam_lr0.001</code> (0,3961); terburuk <code>grid_sgd_lr0.01</code> (0,6183) dan <code>baseline_sgd_lr1</code> (2,3389).	Tabel 1. Learning rate terlalu kecil membuat langkah tidak berarti dalam 5 epoch; learning rate terlalu besar membuat langkah melewati minimum sehingga loss naik.
Trade-off	Kualitas terbaik tidak otomatis tercepat; batch kecil menang dalam satuan epoch, batch besar dalam satuan waktu.	Adam 0,283 s/epoch vs Momentum 0,245 dan SGD 0,226 ($\pm 15\text{--}25\%$ lebih lama untuk selisih loss 0,0157–0,0302). Batch 32: 0,3873 dengan 0,608 s/epoch; batch 512: 0,4477 dengan 0,212 s/epoch (2,9 kali lebih cepat).
Anomali atau hasil gagal	(a) SGD $\eta = 1,0$ tidak konvergen; (b) scheduler menurunkan hasil.	(a) Loss per epoch 2,0318 $\rightarrow$ 2,1560 $\rightarrow$ 1,9747 $\rightarrow$ 2,2695 $\rightarrow$ 2,3389, accuracy 0,3330, <code>grad_norm_mean</code> 1,4769 (tertinggi); gejala muncul di epoch ke-2, ciri langkah melewati lembah. (b) StepLR 0,4047 dan cosine 0,4066 vs 0,3961, karena learning rate dipangkas ke 0,00025 dan mendekati 0 padahal kurva masih menurun.
Generalisasi	Train-validation gap wajar; test sedikit di bawah validation.	Run final: train 0,3466, val 0,3961 (gap 0,0495), test 0,4270/0,8498 (selisih val-test 0,031) – konsisten dengan subset 20% data latih dan 5 epoch, tanpa overfit berat.

6. Jawaban Pertanyaan Modul

- Mengapa batch 32 menghasilkan 1.875 update sedangkan batch 512 hanya 120?** Anggaran epoch menghitung berapa kali *seluruh* data latih dilewati, bukan berapa kali optimizer dipanggil: $\lceil 12000/32 \rceil = 375$ dan $\lceil 12000/512 \rceil = 24$ batch per epoch $\times$ 5 epoch = 1.875 dan 120 update (cocok=True). Keduanya memproses 12.000 contoh yang sama, jadi batch 32 melakukan 15,6 kali lebih banyak langkah parameter. Prediksi terbukti: val loss 0,3873 vs 0,4477 dan 0,6079 vs 0,2117 detik per epoch (2,9 kali).
- Apakah optimizer dengan validation loss terendah juga yang tercepat?** Tidak. Adam $\eta = 0,001$ terendah (0,3961) tetapi paling lambat di antara tiga pemenang (0,283 s/epoch vs Momentum 0,245 dan SGD 0,226); selisih 0,0157–0,0302 dibayar dengan waktu 15–25% lebih lama. Run tercepat `grid_sgd_lr0.1` (1,13 dtk) hanya urutan keempat validation loss, sedangkan dua run terlambat adalah dua run terburuk (0,6183 dan 0,5438).
- Mengapa learning rate terbaik Adam jauh lebih kecil daripada SGD?** SGD memakai langkah $\theta \leftarrow \theta - \eta g$ sehingga langkah bergantung langsung pada besaran gradien dan memerlukan $\eta \approx 10^{-1}$ agar masih berarti dekat minimum; Adam membagi gradien dengan $\sqrt{v}$ dan menambahkan momentum, sehingga langkah efektif $\approx \eta$. Bukti pada grid: Adam $10^{-4} = 0,5438$, $10^{-3} = 0,3961$, $10^{-2} = 0,4794$; SGD terbaik justru di 10^{-1} (0,4263).
- Apa yang terjadi saat learning rate terlalu besar, dan pada epoch beberapa gejalanya terlihat?** Pada SGD $\eta = 1,0$ validation loss bergerigi lalu naik; gejala pertama tampak pada **epoch ke-2** (2,0318 $\rightarrow$ 2,1560) dan berakhir di 2,3389 dengan accuracy 0,3330 – ciri langkah yang melewati lembah sehingga parameter memantul, bukan sekadar konvergen lambat.
- Apakah scheduler memperbaiki hasil pada anggaran epoch yang sama?** Tidak. Pada batch 128 dan 5 epoch: tanpa scheduler 0,3961, StepLR(step_size=2, gamma=0.5) 0,4047, CosineAnnealingLR(T_max=5) 0,4066. Learning rate dipangkas ke 0,00025 dan mendekati 0 padahal kurva masih menurun tajam, sehingga scheduler hanya membuang sisa anggaran.
- Batch size mana yang direkomendasikan bila anggaran diukur dalam waktu?** Batch 128: val loss 0,3961 (hanya 0,0088 di atas batch 32) dengan 0,331 s/epoch; batch 32 mahal (0,608 s/epoch) dan batch 512 kalah jelas (0,4477). Pada anggaran 3,04 detik, batch 512 dapat ± 14 epoch dan batch 128 ± 9 epoch, sehingga pada anggaran panjang batch besar harus diberi lebih banyak epoch.

7. Kesimpulan dan Keterbatasan

7.1 Kesimpulan

Adam dengan learning rate 0,001, batch 128, dan tanpa scheduler dipilih sebagai konfigurasi terbaik karena memberi validation loss terendah 0,3961 dan validation accuracy 0,8560 – unggul 0,0202 atas Momentum (0,4118) dan 0,0302 atas SGD (0,4263) – serta test accuracy 0,8498 pada evaluasi tunggal. Pada anggaran 5 epoch, scheduler tidak membantu (0,4047 dan 0,4066) dan batch 32 hanya unggul 0,0088 atas batch 128 tetapi membutuhkan 1.875 update serta 2,9 kali lebih lama per epoch daripada batch 512. Bila batas diukur dalam epoch maka batch 32 sedikit lebih baik, bila diukur dalam detik maka batch 128 adalah kompromi terbaik antara kualitas (0,3961) dan biaya per epoch.

7.2 Keterbatasan

– Hanya 12.000 citra (20% data latih) dan 5 epoch, sehingga kesimpulan berlaku pada anggaran pendek dan performa penuh Fashion-MNIST dapat berbeda. – Satu seed (24) tanpa pengulangan, sehingga selisih Adam–SGD 0,0202 belum memiliki signifikansi statistik. – Tiap optimizer hanya diuji pada tiga learning rate dan satu arsitektur FNN (tanpa CNN, weight decay, atau augmentasi), sehingga konfigurasi terbaik belum tentu global. – Runtime diukur sekali di CPU dengan beban sistem tidak terkendali (pada konfigurasi identik terekam 0,33 dan 0,44 detik per epoch), sehingga urutan kecepatan bukan benchmark baku.

7.3 Tindak lanjut

Eksperimen paling bernilai adalah menjalankan ulang ketiga optimizer pada anggaran 100–200 epoch dengan scheduler yang sama, karena manfaat StepLR/cosine baru terasa setelah langkah kecil diperlukan untuk menyempurnakan sekitar minimum; pengulangan 3–5 seed pada Adam vs SGD juga diperlukan untuk menguji apakah selisih 0,0202 konsisten.

Referensi

- PyTorch Contributors. *torch.optim (SGD, Adam, StepLR, CosineAnnealingLR)*. Diakses 2026-09-30. <https://pytorch.org/docs/stable/optim.html>
- PyTorch Contributors. *torchvision.datasets.FashionMNIST*. Diakses 2026-09-30. <https://pytorch.org/vision/main/generated/torchvision.datasets.FashionMNIST.html>
- Kingma, D. P. & Ba, J. 2015. Adam: A Method for Stochastic Optimization. ICLR 2015. arXiv:1412.6980.
- Pedregosa, F. dkk. 2011. Scikit-learn: Machine Learning in Python. JMLR 12:2825–2830.

Pernyataan Orisinalitas. Saya menyatakan bahwa kode, eksperimen, analisis, dan laporan ini merupakan pekerjaan individual; seluruh sumber eksternal telah dicantumkan. Kemiripan hasil akibat seed atau data yang sama tidak membenarkan penyalinan notebook maupun analisis.

Nama: Natasya Amavisca **NIM:** 123450024

Tanggal: 2026-10-03